

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA0201

COURSE NAME: C Programming

COURSE OUTCOME COVERED

CO1: Construct structured C programs using constants, variables, data types, and preprocessor directives to address basic computational tasks. (BL2,BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

I, ASHIRA BRIDGET SUSAN S (192524316) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Experiment 1: Comprehensive C Program Design Develop a C program for a university admission system that accepts a student's name, registration number, marks in five subjects, and category (General/Reserved). The program should use variables of different data types, symbolic constants for pass marks, arithmetic and relational operators to calculate percentage and eligibility, and decision-making statements to determine admission status. Include appropriate preprocessor directives and comments explaining the structure of the C program.

Aim

To develop a C program for a university admission system that accepts a student's details, calculates percentage marks, and determines admission eligibility based on pass marks and category.

Algorithm

1. Start the program.
2. Accept the student's name and registration number.
3. Read marks of five subjects.
4. Read the student's category (General/Reserved).
5. Calculate the total and percentage.
6. Check whether the student has passed all subjects.
7. If the student belongs to:
 - General category → percentage must be $\geq 60\%$.
 - Reserved category → percentage must be $\geq 50\%$.
8. Display the admission status.
9. Stop the program.

C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
/* Symbolic Constants */
```

```
#define PASS_MARK 40
```

```
#define GENERAL_PERCENTAGE 60
```

```
#define RESERVED_PERCENTAGE 50
```

```
int main()
{
    /* Variable Declarations */
    char name[50];
    char regNo[20];
    char category[20];

    int mark1, mark2, mark3, mark4, mark5;
    int total;
    float percentage;

    /* Input Student Details */
    printf("Enter Student Name: ");
    fgets(name, sizeof(name), stdin);

    printf("Enter Registration Number: ");
    scanf("%s", regNo);

    printf("Enter Marks in 5 Subjects:\n");
    scanf("%d %d %d %d %d",
        &mark1, &mark2, &mark3,
        &mark4, &mark5);

    printf("Enter Category (General/Reserved): ");
    scanf("%s", category);

    /* Calculate Total and Percentage */
    total = mark1 + mark2 + mark3 + mark4 + mark5;
    percentage = (total / 500.0) * 100;
```

```

/* Display Student Details */

printf("\n----- Student Details ----- \n");

printf("Name : %s", name);

printf("Registration Number : %s\n", regNo);

printf("Percentage : %.2f%%\n", percentage);


/* Check Pass Status */

if (mark1 >= PASS_MARK &&
    mark2 >= PASS_MARK &&
    mark3 >= PASS_MARK &&
    mark4 >= PASS_MARK &&
    mark5 >= PASS_MARK)
{
    /* Admission Eligibility */

    if (strcmp(category, "General") == 0)
    {
        if (percentage >= GENERAL_PERCENTAGE)
        {
            printf("Admission Status : ELIGIBLE\n");
        }
        else
        {
            printf("Admission Status : NOT ELIGIBLE\n");
        }
    }
    else if (strcmp(category, "Reserved") == 0)
    {

```

```
    if (percentage >= RESERVED_PERCENTAGE)
    {
        printf("Admission Status : ELIGIBLE\n");
    }
    else
    {
        printf("Admission Status : NOT ELIGIBLE\n");
    }
}
else
{
    printf("Invalid Category Entered.\n");
}
}
else
{
    printf("Admission Status : NOT ELIGIBLE (Failed in one or more subjects)\n");
}

return 0;
}
```

Sample Input

Enter Student Name: Ashira Susan

Enter Registration Number: 23AIDS101

Enter Marks in 5 Subjects:

85 90 78 88 92

Enter Category (General/Reserved): General

Sample Output

----- Student Details -----

Name : Ashira Susan

Registration Number : 23AIDS101

Percentage : 86.60%

Admission Status : ELIGIBLE

Experiment 2: Operator Precedence and Expression Evaluation Write a C program that accepts three integer values and evaluates at least five complex expressions involving arithmetic, relational, logical, increment/decrement, and conditional operators. Predict the output manually by applying operator precedence and associativity rules before executing the program. Compare the predicted and actual outputs and justify any differences observed.

Aim

To write a C program that accepts three integer values and evaluates complex expressions involving arithmetic, relational, logical, increment/decrement, and conditional operators and to study operator precedence and associativity rules.

Algorithm

1. Start the program.
2. Read three integer values from the user.
3. Evaluate expressions using:
 - Arithmetic operators (+, -, *, /, %)
 - Relational operators (>, <, ==)
 - Logical operators (&&, ||, !)
 - Increment/Decrement operators (++ , --)
 - Conditional operator (?:)
4. Display the results of all expressions.
5. Stop the program.

C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, c;
```

```

/* Taking input from the user */
printf("Enter three integers (a b c): ");
scanf("%d %d %d", &a, &b, &c);

printf("\n----- Expression Evaluation ----- \n");

/* Arithmetic operators */
printf("1. (a + b * c) = %d\n", (a + b * c));

/* Relational operator */
printf("2. (a > b) = %d\n", (a > b));

/* Logical operators */
printf("3. (a > b && b < c) = %d\n",
      (a > b && b < c));

/* Increment and Decrement operators */
printf("4. (++a + --b) = %d\n",
      (++a + --b));

/* Conditional (Ternary) operator */
printf("5. (a > c ? a : c) = %d\n",
      (a > c ? a : c));

return 0;
}

```

Sample Input

Enter three integers (a b c): 10 5 20

Manual Prediction of Output

Initially,

$a = 10$

$b = 5$

$c = 20$

Expression 1

$(a + b * c)$

$= 10 + (5 \times 20)$

$= 10 + 100$

$= 110$

Output:

110

* has higher precedence than +.

Expression 2

$(a > b)$

$= (10 > 5)$

$= \text{True}$

Output = 1

In C, 1 means True and 0 means False.

Expression 3

$(a > b \ \&\& \ b < c)$

$= (10 > 5) \ \&\& \ (5 < 20)$

$= \text{True} \ \&\& \ \text{True}$

= True

Output = 1

Relational operators are evaluated before logical &&.

Expression 4

(++a + --b)

a = 10 → ++a = 11

b = 5 → --b = 4

11 + 4 = 15

Output:

15

Notice that a and b are permanently modified after this expression.

Current values become:

a = 11

b = 4

c = 20

Expression 5

(a > c ? a : c)

(11 > 20)?

False

Output = c = 20

Output:

Actual Output

----- Expression Evaluation -----

1. $(a + b * c) = 110$

2. $(a > b) = 1$

3. $(a > b \ \&\& \ b < c) = 1$

4. $(++a + --b) = 15$

5. $(a > c ? a : c) = 20$

Experiment 3: Employee Payroll Management System
Design and implement a payroll management system using C that calculates gross salary, tax deductions, and net salary for employees. Use constants for tax slabs, variables of appropriate data types, nested if-else statements for tax computation, and switch statements for employee categories. The program should repeatedly process employee records until the user chooses to exit using looping constructs.

Experiment 3: Employee Payroll Management System

Aim

To design and implement an Employee Payroll Management System in C that calculates gross salary, tax deductions, and net salary using constants, variables, nested if-else statements, switch statements, and looping constructs.

Algorithm

1. Start the program.
2. Repeat the following steps until the user chooses to exit:
 - Read the employee's name, ID, basic salary, and category.
 - Calculate allowances based on the employee category using switch-case.
 - Calculate gross salary.
 - Compute tax deductions using nested if-else statements.
 - Calculate net salary.
 - Display the payroll details.
 - Ask the user whether they want to process another employee.
3. If the user chooses to exit, terminate the program.
4. Stop the program.

C Program

```
#include <stdio.h>
```

```
/* Symbolic Constants */
```

```
#define TAX_SLAB1 0.05
```

```
#define TAX_SLAB2 0.10
```

```
#define TAX_SLAB3 0.20
```

```
int main()
{
    int empID, category, choice = 1;
    char name[50];

    float basicSalary;
    float allowance = 0;
    float grossSalary;
    float tax = 0;
    float netSalary;

    while (choice == 1)
    {
        printf("\n===== EMPLOYEE PAYROLL SYSTEM =====\n");

        printf("Enter Employee Name: ");
        scanf(" %[^\\n]", name);

        printf("Enter Employee ID: ");
        scanf("%d", &empID);

        printf("Enter Basic Salary: ");
        scanf("%f", &basicSalary);

        printf("\nEmployee Categories\n");
        printf("1. Manager (30%% Allowance)\n");
        printf("2. Developer (20%% Allowance)\n");
        printf("3. Clerk (10%% Allowance)\n");
```

```
printf("Enter Category (1-3): ");
scanf("%d", &category);

/* Calculate Allowances */
switch (category)
{
    case 1:
        allowance = basicSalary * 0.30;
        break;

    case 2:
        allowance = basicSalary * 0.20;
        break;

    case 3:
        allowance = basicSalary * 0.10;
        break;

    default:
        printf("Invalid Category!\n");
        allowance = 0;
}

/* Gross Salary Calculation */
grossSalary = basicSalary + allowance;

/* Tax Calculation using Nested If-Else */

if (grossSalary <= 30000)
```

```

{
    tax = grossSalary * TAX_SLAB1;
}
else
{
    if (grossSalary <= 60000)
    {
        tax = grossSalary * TAX_SLAB2;
    }
    else
    {
        tax = grossSalary * TAX_SLAB3;
    }
}

/* Net Salary */
netSalary = grossSalary - tax;

/* Display Payroll Details */

printf("\n----- PAYROLL DETAILS ----- \n");

printf("Employee Name : %s\n", name);
printf("Employee ID   : %d\n", empID);
printf("Basic Salary   : %.2f\n", basicSalary);
printf("Allowance      : %.2f\n", allowance);
printf("Gross Salary    : %.2f\n", grossSalary);
printf("Tax Deduction   : %.2f\n", tax);
printf("Net Salary      : %.2f\n", netSalary);

```

```

/* Continue or Exit */

printf("\nDo you want to process another employee?\n");
printf("Enter 1 for YES or 0 for NO: ");
scanf("%d", &choice);
}

printf("\nThank You for Using Payroll Management System!\n");

return 0;
}

```

Sample Input

Enter Employee Name: Ashira

Enter Employee ID: 101

Enter Basic Salary: 50000

Enter Category: 2

Calculation

Basic Salary = 50000

Developer Allowance = 20%

= 10000

Gross Salary = 50000 + 10000

= 60000

Tax = 10%

= 6000

$$\begin{aligned}\text{Net Salary} &= 60000 - 6000 \\ &= 54000\end{aligned}$$

Sample Output

===== EMPLOYEE PAYROLL SYSTEM =====

Enter Employee Name: Ashira

Enter Employee ID: 101

Enter Basic Salary: 50000

Employee Categories

1. Manager (30% Allowance)
2. Developer (20% Allowance)
3. Clerk (10% Allowance)

Enter Category (1-3): 2

----- PAYROLL DETAILS -----

Employee Name : Ashira

Employee ID : 101

Basic Salary : 50000.00

Allowance : 10000.00

Gross Salary : 60000.00

Tax Deduction : 6000.00

Net Salary : 54000.00

Do you want to process another employee?

Enter 1 for YES or 0 for NO: 0

Thank You for Using Payroll Management System!

Experiment 4: Intelligent Number Analysis System
Develop a menu-driven C application that performs multiple operations on a given number, such as checking whether it is prime, palindrome, Armstrong, perfect, or strong. Use switch-case, nested branching statements, and loops wherever applicable. The program should continue execution until the user selects the termination option and must handle invalid choices gracefully.

Aim

To develop a menu-driven C program that performs multiple operations on a given number such as checking whether it is Prime, Palindrome, Armstrong, Perfect, or Strong using switch-case, nested decision-making statements, and looping constructs.

Algorithm

1. Start the program.
2. Display a menu of operations.
3. Accept the user's choice.
4. Read a number whenever required.
5. Perform the selected operation:
 - Prime Number Check
 - Palindrome Number Check
 - Armstrong Number Check
 - Perfect Number Check
 - Strong Number Check
6. Display the result.
7. Continue displaying the menu until the user selects Exit.
8. Handle invalid menu choices appropriately.
9. Stop the program.

C Program

```
#include <stdio.h>
```

```

int main()
{
    int choice, num, i;
    int temp, remainder, sum;

    do
    {
        printf("\n===== INTELLIGENT NUMBER ANALYSIS SYSTEM =====\n");
        printf("1. Check Prime Number\n");
        printf("2. Check Palindrome Number\n");
        printf("3. Check Armstrong Number\n");
        printf("4. Check Perfect Number\n");
        printf("5. Check Strong Number\n");
        printf("6. Exit\n");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            /* PRIME NUMBER */
            case 1:
            {
                int isPrime = 1;

                printf("Enter a number: ");
                scanf("%d", &num);

                if(num <= 1)

```

```

    {
        isPrime = 0;
    }
    else
    {
        for(i = 2; i < num; i++)
        {
            if(num % i == 0)
            {
                isPrime = 0;
                break;
            }
        }
    }

    if(isPrime)
        printf("%d is a Prime Number.\n", num);
    else
        printf("%d is NOT a Prime Number.\n", num);

    break;
}

```

/* PALINDROME NUMBER */

case 2:

```

{
    int reverse = 0;

```

```

printf("Enter a number: ");
scanf("%d", &num);

temp = num;

while(temp != 0)
{
    remainder = temp % 10;
    reverse = reverse * 10 + remainder;
    temp /= 10;
}

if(reverse == num)
    printf("%d is a Palindrome Number.\n", num);
else
    printf("%d is NOT a Palindrome Number.\n", num);

break;
}

/* ARMSTRONG NUMBER */
case 3:
{
    printf("Enter a number: ");
    scanf("%d", &num);

    temp = num;
    sum = 0;

```

```

while(temp != 0)
{
    remainder = temp % 10;
    sum += remainder * remainder * remainder;
    temp /= 10;
}

if(sum == num)
    printf("%d is an Armstrong Number.\n", num);
else
    printf("%d is NOT an Armstrong Number.\n", num);

break;
}

```

```

/* PERFECT NUMBER */

```

```

case 4:

```

```

{
    printf("Enter a number: ");
    scanf("%d", &num);

```

```

    sum = 0;

```

```

    for(i = 1; i < num; i++)

```

```

    {
        if(num % i == 0)
        {

```

```
        sum += i;
    }
}

if(sum == num)
    printf("%d is a Perfect Number.\n", num);
else
    printf("%d is NOT a Perfect Number.\n", num);

break;
}
```

```
/* STRONG NUMBER */
case 5:
{
    printf("Enter a number: ");
    scanf("%d", &num);

    temp = num;
    sum = 0;

    while(temp != 0)
    {
        int fact = 1;

        remainder = temp % 10;

        for(i = 1; i <= remainder; i++)
```



```

        {
            fact *= i;
        }

        sum += fact;
        temp /= 10;
    }

    if(sum == num)
        printf("%d is a Strong Number.\n", num);
    else
        printf("%d is NOT a Strong Number.\n", num);

    break;
}

/* EXIT */

case 6:
    printf("Program Terminated Successfully.\n");
    break;

/* INVALID CHOICE */

default:
    printf("Invalid Choice! Please Try Again.\n");
}

} while(choice != 6);

```

```
    return 0;  
}
```

Sample Output

===== INTELLIGENT NUMBER ANALYSIS SYSTEM =====

1. Check Prime Number
2. Check Palindrome Number
3. Check Armstrong Number
4. Check Perfect Number
5. Check Strong Number
6. Exit

Enter your choice: 1

Enter a number: 13

13 is a Prime Number.

===== INTELLIGENT NUMBER ANALYSIS SYSTEM =====

Enter your choice: 2

Enter a number: 121

121 is a Palindrome Number.

===== INTELLIGENT NUMBER ANALYSIS SYSTEM =====

Enter your choice: 3

Enter a number: 153

153 is an Armstrong Number.

===== INTELLIGENT NUMBER ANALYSIS SYSTEM =====

Enter your choice: 4

Enter a number: 28

28 is a Perfect Number.

===== INTELLIGENT NUMBER ANALYSIS SYSTEM =====

Enter your choice: 5

Enter a number: 145

145 is a Strong Number.

===== INTELLIGENT NUMBER ANALYSIS SYSTEM =====

Enter your choice: 6

Program Terminated Successfully.

Experiment 5: Traffic Signal and Vehicle Monitoring System
Create a C program to simulate a smart traffic control system. The program should accept vehicle count, signal status, and emergency vehicle information as input. Use preprocessor directives for defining signal states, decision-making constructs for traffic management, and loops to monitor traffic data continuously. The system should prioritize emergency vehicles, display appropriate traffic instructions, and generate a summary report at the end of execution

Aim

To develop a C program that simulates a smart traffic control system by monitoring vehicle count, signal status, and emergency vehicle information. The program prioritizes emergency vehicles, provides traffic instructions, and generates a summary report using preprocessor directives, decision-making statements, and looping constructs.

Algorithm

1. Start the program.
2. Define symbolic constants for traffic signal states using preprocessor directives.
3. Continuously accept:
 - Vehicle count
 - Current signal status
 - Emergency vehicle information
4. If an emergency vehicle is detected:
 - Give it the highest priority.
 - Change the signal to GREEN.
5. Otherwise:
 - Manage traffic based on the vehicle count and signal status.
6. Display appropriate traffic instructions.
7. Maintain a count of the total vehicles monitored.
8. Ask the user whether they wish to continue monitoring.
9. Display a summary report when the user exits.
10. Stop the program.

```
#include <stdio.h>
```

```
/* Preprocessor Directives */

#define RED 0

#define YELLOW 1

#define GREEN 2


int main()
{
    int vehicleCount;

    int signalStatus;

    int emergencyVehicle;

    int choice = 1;


    int totalVehicles = 0;

    int emergencyCount = 0;

    int monitoringCycles = 0;


    while(choice == 1)
    {
        printf("\n===== SMART TRAFFIC CONTROL SYSTEM =====\n");


        printf("Enter Vehicle Count: ");

        scanf("%d", &vehicleCount);


        printf("\nSignal Status:\n");

        printf("0 - RED\n");

        printf("1 - YELLOW\n");

        printf("2 - GREEN\n");

        printf("Enter Signal Status: ");
```

```
scanf("%d", &signalStatus);

printf("Is there an Emergency Vehicle?\n");
printf("Enter 1 for YES or 0 for NO: ");
scanf("%d", &emergencyVehicle);

monitoringCycles++;
totalVehicles += vehicleCount;

/* Emergency Vehicle Handling */

if(emergencyVehicle == 1)
{
    emergencyCount++;
    signalStatus = GREEN;

    printf("\n*** EMERGENCY VEHICLE DETECTED ***\n");
    printf("Signal changed to GREEN.\n");
    printf("Please give way immediately.\n");
}
else
{
    /* Traffic Management */

    if(vehicleCount > 50)
    {
        printf("\nHeavy Traffic Detected.\n");

        if(signalStatus == RED)
```

```
{  
    printf("Remain Stopped.\n");  
}  
else if(signalStatus == YELLOW)  
{  
    printf("Prepare to Move Carefully.\n");  
}  
else if(signalStatus == GREEN)  
{  
    printf("Proceed Slowly and Maintain Lane Discipline.\n");  
}  
else  
{  
    printf("Invalid Signal Status!\n");  
}  
}  
else  
{  
    printf("\nNormal Traffic Conditions.\n");  
  
switch(signalStatus)  
{  
    case RED:  
        printf("STOP.\n");  
        break;  
  
    case YELLOW:  
        printf("WAIT. Signal is about to change.\n");  
        break;
```

```

        case GREEN:

            printf("GO. Drive Safely.\n");

            break;


        default:

            printf("Invalid Signal Status!\n");

        }

    }

}

/* Continue Monitoring */

printf("\nContinue Monitoring?\n");
printf("Enter 1 for YES or 0 for NO: ");
scanf("%d", &choice);

}

/* Summary Report */

printf("\n===== TRAFFIC SUMMARY REPORT =====\n");

printf("Total Monitoring Cycles : %d\n", monitoringCycles);
printf("Total Vehicles Monitored: %d\n", totalVehicles);
printf("Emergency Vehicles      : %d\n", emergencyCount);

printf("\nTraffic Monitoring Successfully Terminated.\n");

```



```
    return 0;  
}
```

Sample Input 1

Enter Vehicle Count: 65

Enter Signal Status: 0

Emergency Vehicle: 0

Sample Output 1

Heavy Traffic Detected.

Remain Stopped.

Sample Input 2

Enter Vehicle Count: 25

Enter Signal Status: 2

Emergency Vehicle: 0

Sample Output 2

Normal Traffic Conditions.

GO. Drive Safely.

Sample Input 3

Enter Vehicle Count: 40

Enter Signal Status: 0

Emergency Vehicle: 1

Sample Output 3

*** EMERGENCY VEHICLE DETECTED ***

Signal changed to GREEN.

Please give way immediately.

Sample Summary Report

===== TRAFFIC SUMMARY REPORT =====

Total Monitoring Cycles : 3

Total Vehicles Monitored: 130

Emergency Vehicles : 1

Traffic Monitoring Successfully Terminated.